

Test Analytics

Test Analytics offers data on test run times, failure rates, and identifies flaky tests to help decrease the risk of deployment failures and make it easier to ship new features quickly.

Select a setup option

Using GitHub Actions



Choose this option, if you have been using GitHub actions to run your CI.

Using Codecov's CLI



Choose this option, if you have been using Codecov's CLI to upload coverage reports.

Step 1: Output a JUnit XML file in your CI

Select the framework below to generate a JUnit XML file that contains the results of your test run.

Pytest

Vitest

Jest

PHPunit

```
vitest --reporter=junit --outputFile=test-report.junit.xml
```



Step 2: Add the script `codecov/test-results-action@v1` to your CI YAML file.

In your CI YAML file, add below scripts to the end of your test run.

```
- name: Upload test results to Codecov
  if: ${ !cancelled() }}
  uses: codecov/test-results-action@v1
  with:
    token: ${ secrets.CODECOV_TOKEN }}
```



This action will download the Codecov CLI, and upload the `junit.xml` file generated in the previous step to Codecov.

Your final `ci.yaml` file for a project using `pytest` could look something like this:



Step 3: Run your test suite

You can inspect the workflow logs to see if the call to Codecov succeeded.

```
98 info - 2024-02-09 13:40:22,646 -- Found 1 test_results files to upload
99 info - 2024-02-09 13:40:22,646 -- > /home/runner/work/mcos/mcos/thebeast/mcos.junit.xml
100 info - 2024-02-09 13:40:22,728 -- Process Upload complete
```

Run your tests as usual. You need to have some failed tests to view the failed tests report.

Here are examples of failed test reports in PR comments. Comment generation may take time.



Step 4: View results and insights

After the test run completion, you'll be able to see the failed tests result on the following areas:

- GitHub pull request comment
- Failed tests dashboard here.

Visit our guide to [learn more](#)  about test ingestion.

